

**PSEUDOCODE WRITING**[← Back](#)

QUESTIONS

Q1 ✓
Student Management System
[BL3 – Apply]
10 marks**Q2** ✓
Library Book Management [BL3
– Apply]
10 marks**Q3** ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4** ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5** ✓
5. Vehicle Registration System**Q1 Student Management System [BL3 – Apply]**

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

START

CLASS Student

PRIVATE:

name

rollNo

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
METHOD setStudentDetails(name, rollNo, marks[])
```

```
  SET this.name = name  
  SET this.rollNo = rollNo  
  SET this.marks[] = marks[]  
END METHOD
```

```
METHOD calculateAverage()
```

```
  SET total = 0  
  FOR each mark IN marks[]  
    total = total + mark  
  END FOR  
  SET average = total / LENGTH(marks[])  
  RETURN average  
END METHOD
```

```
METHOD displayResult()
```

```
  SET average = calculateAverage()  
  PRINT "Student Name: " + name  
  PRINT "Roll Number: " + rollNo  
  PRINT "Average Marks: " + average  
  IF average >= 50 THEN  
    PRINT "Result: PASS"  
  ELSE
```

```
        PRINT "Result: FAIL"
    END IF
END METHOD

CLASS Student
    PRIVATE
        name (String)
        rollNo (Integer)
        marks[] (Array of Float)

    METHOD setStudentDetails(name, rollNo, marks[])
        SET this.name = name
        SET this.rollNo = rollNo
        SET this.marks[] = marks[]
    END METHOD

    METHOD calculateAverage()
        SET total = 0
        FOR each mark IN marks[]
            total = total + mark
        END FOR
        SET average = total / LENGTH(marks[])
        RETURN average
    END METHOD
```

```
METHOD displayResult()
    SET average = calculateAverage()
    PRINT "Student Name: " + name
    PRINT "Roll Number: " + rollNo
    PRINT "Average Marks: " + average
    IF average >= 50 THEN
        PRINT "Result: PASS"
    ELSE
        PRINT "Result: FAIL"
    END IF
END METHOD

END CLASS

BEGIN MAIN
    CREATE object student OF CLASS Student
    INPUT name, rollNo, marks[]
    CALL student.setStudentDetails(name, rollNo, marks[])
    CALL student.displayResult()
END MAIN OF CLASS Student
INPUT name, rollNo, marks[]
CALL student.setStudentDetails(name, rollNo, marks[])
CALL student.displayResult()
END MAIN
```

Q2 Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

START

CREATE Book with title, author, isbn, isAvailable

CREATE Book with title, author, isbn, isAvailable

CREATE Book with title, author, isbn, isAvailable

PRINT details of all 3 books using getters

CALL issueBook() on first book

CALL issueBook() on second book

PRINT details of all 3 books again

CALL returnBook() on first book

PRINT details of all 3 books final

END

Book class:

title, author, isbn, isAvailable are private

CONSTRUCTOR(title, author, isbn, isAvailable)

```
SET this.title = title
SET this.author = author
SET this.isbn = isbn
SET this.isAvailable = isAvailable
```

```
GETTER getTitle()
RETURN title
```

```
GETTER getAuthor()
RETURN author
```

```
GETTER getIsbn()
RETURN isbn
```

```
GETTER getIsAvailable()
RETURN isAvailable
```

```
METHOD issueBook()
IF isAvailable is true THEN
    SET isAvailable = false
ELSE
    DO nothing (or show message)
```

```
METHOD returnBook()
IF isAvailable is false THEN
    SET isAvailable = true
```


ELSE

DO nothing (or show message)

METHOD displayDetails()

PRINT getTitle, getAuthor, getIsbn, getIsAvailable

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to: •

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)

- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to: •

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)

- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

PSEUDOCODE: Employee Salary Calculation (Encapsulation + Data Hiding)

```
CLASS Employee
  PRIVATE empld, name, basicPay, designation
  CONSTRUCTOR(id, nm, bp, des)
    empld = id
    name = nm
    basicPay = bp
    designation = des
  END CONSTRUCTOR
  PRIVATE METHOD grossSalary()
    hra = 0.20 * basicPay
    da = 0.10 * basicPay
    RETURN basicPay + hra + da
  END METHOD
  PRIVATE METHOD netSalary()
    gross = grossSalary()
    pf = 0.12 * gross
    tax = 0.05 * gross
    RETURN gross - (pf + tax)
  END METHOD
  PUBLIC METHOD displayPaySlip()
    gross = grossSalary()
    net = netSalary()
```

```
PRINT "Empld:", empld, "Name:", name, "Designation:", designation
PRINT "Gross:", gross, "Net:", net
END METHOD
END CLASS
BEGIN
  e = NEW Employee(192424282, "B C YUGESH", 50000, "Employee")
  e.displayPaySlip()
END
```

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable `calculateArea()` method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call `calculateArea()` for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
CLASS Shape
  METHOD calculateArea()
    RETURN 0
END CLASS

CLASS Circle EXTENDS Shape
  PRIVATE radius
  CONSTRUCTOR Circle(r)
    radius = r
  END CONSTRUCTOR

  METHOD calculateArea()
    RETURN 3.14159 * radius * radius
  END METHOD
END CLASS

CLASS Rectangle EXTENDS Shape
  PRIVATE length
  PRIVATE breadth
  CONSTRUCTOR Rectangle(l, b)
    length = l
    breadth = b
  END CONSTRUCTOR
```

```
METHOD calculateArea()  
    RETURN length * breadth  
END METHOD  
END CLASS
```

```
CLASS Triangle EXTENDS Shape  
    PRIVATE base  
    PRIVATE height  
    CONSTRUCTOR Triangle(b, h)  
        base = b  
        height = h  
    END CONSTRUCTOR
```

```
METHOD calculateArea()  
    RETURN 0.5 * base * height  
END METHOD  
END CLASS
```

```
PROGRAM Main  
    Shape s1 = NEW Circle(7)  
    Shape s2 = NEW Rectangle(5, 10)  
    Shape s3 = NEW Triangle(6, 8)
```



```
PRINT "Circle Area: " + s1.calculateArea()  
PRINT "Rectangle Area: " + s2.calculateArea()  
PRINT "Triangle Area: " + s3.calculateArea()  
END PROGRAM
```

Q5 5. Vehicle Registration System [BL3 – Apply]

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

PSEUDOCODE: Vehicle Registration System

CLASS Vehicle

DATA regNo

DATA owner

DATA fuelType

CONSTRUCTOR Vehicle(regNo, owner, fuelType)

SET self.regNo = regNo

SET self.owner = owner

SET self.fuelType = fuelType

END CONSTRUCTOR

METHOD displayDetails()

PRINT "Registration No: " + regNo

PRINT "Owner: " + owner

PRINT "Fuel Type: " + fuelType

END METHOD

END CLASS

```
CLASS TwoWheeler EXTENDS Vehicle
```

```
    DATA engineCapacity
```

```
    CONSTRUCTOR TwoWheeler(regNo, owner, fuelType, engineCapacity)
```

```
        CALL Vehicle.CONSTRUCTOR(regNo, owner, fuelType)
```

```
        SET self.engineCapacity = engineCapacity
```

```
    END CONSTRUCTOR
```

```
    METHOD displayDetails()
```

```
        CALL Vehicle.displayDetails()
```

```
        PRINT "Engine Capacity: " + engineCapacity + " cc"
```

```
    END METHOD
```

```
END CLASS
```

```
CLASS FourWheeler EXTENDS Vehicle
```

```
    DATA seatingCapacity
```

```
    DATA acStatus
```

```
    CONSTRUCTOR FourWheeler(regNo, owner, fuelType, seatingCapacity, acStatus)
```

```
        CALL Vehicle.CONSTRUCTOR(regNo, owner, fuelType)
```

```
    SET self.seatingCapacity = seatingCapacity
    SET self.acStatus = acStatus
END CONSTRUCTOR

METHOD displayDetails()
    CALL Vehicle.displayDetails()
    PRINT "Seating Capacity: " + seatingCapacity + " persons"
    IF acStatus == TRUE THEN
        PRINT "AC Status: Available"
    ELSE
        PRINT "AC Status: Not Available"
    END IF
END METHOD
END CLASS

PROGRAM Main
    DECLARE v1 AS Vehicle
    DECLARE v2 AS Vehicle

    SET v1 = NEW TwoWheeler("TN09AB1234", "B. Yugesh", "Petrol", 150.5)
```

```
SET v2 = NEW FourWheeler("TN07CD5678", "Yugesh B c", "Diesel", 5, TRUE)
```

```
PRINT "Two Wheeler Details"
```

```
CALL v1.displayDetails()
```

```
PRINT "Four Wheeler Details"
```

```
CALL v2.displayDetails()
```

```
END PROGRAM
```

Q6 6. Bank Account Hierarchy [BL4 – Analyze]

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
CLASS BankAccount
  ATTRIBUTES
    accNo
    balance
    holder

  CONSTRUCTOR BankAccount(accNo, balance, holder)
    SET this.accNo = accNo
    SET this.balance = balance
    SET this.holder = holder

  METHOD deposit(amount)
    IF amount > 0 THEN
      balance = balance + amount
    ENDIF

  METHOD withdraw(amount)
    IF amount > 0 AND amount <= balance THEN
      balance = balance - amount
      RETURN "SUCCESS"
    ELSE
      RETURN "FAILED"
    ENDIF
```


ENDCLASS

CLASS SavingsAccount EXTENDS BankAccount

ATTRIBUTES

interestRate

CONSTRUCTOR SavingsAccount(accNo, balance, holder, interestRate)

CALL super.BANKAccount(accNo, balance, holder)

SET this.interestRate = interestRate

METHOD applyInterest()

balance = balance + (balance * interestRate)

METHOD withdraw(amount) // override

IF amount > 0 AND amount <= balance THEN

balance = balance - amount

RETURN "SUCCESS"

ELSE

RETURN "FAILED"

ENDIF

ENDCLASS

```

CONSTRUCTOR CurrentAccount(accNo, balance, holder, overdraftLimit)
    CALL super.BANKAccount(accNo, balance, holder)
    SET this.overdraftLimit = overdraftLimit
METHOD withdraw(amount)
    IF amount > 0 THEN
        IF (balance - amount) >= (-overdraftLimit) THEN
            balance = balance - amount
            RETURN "SUCCESS"
        ELSE
            RETURN "FAILED"
        ENDIF
    ELSE
        RETURN "FAILED"
    ENDIF
ENDCLASS
PROCEDURE main()
    acc1 = NEW SavingsAccount("SA1001", 10000, "Yugesh B C", 0.05)
    acc2 = NEW CurrentAccount("CA2001", 3000, "Yugesh B C", 5000)

    PRINT acc1.withdraw(12000)
    PRINT acc1.withdraw(5000)
    PRINT acc2.withdraw(7000)
    PRINT acc2.withdraw(12000)
ENDPROCEDURE

```

Q7 7. Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
CLASS Patient
  PRIVATE patientId
  PRIVATE name
  PRIVATE age
  PRIVATE diagnosis
  PRIVATE prescription

  CONSTRUCTOR Patient(patientId, name, age)
    SET this.patientId = patientId
    SET this.name = name
    SET this.age = age

  METHOD setDiagnosis(text)
    SET diagnosis = text

  METHOD setPrescription(text)
    SET prescription = text

  METHOD getDiagnosis()
    RETURN diagnosis

  METHOD getPrescription()
    RETURN prescription
```

```
METHOD generateBill()  
  ABSTRACT
```

```
CLASS InPatient EXTENDS Patient
```

```
  PRIVATE wardNumber  
  PRIVATE admissionDate  
  PRIVATE dailyCharges
```

```
CONSTRUCTOR InPatient(patientId, name, age, wardNumber, admissionDate, dailyCharges)
```

```
  CALL Patient CONSTRUCTOR with patientId, name, age  
  SET this.wardNumber = wardNumber  
  SET this.admissionDate = admissionDate  
  SET this.dailyCharges = dailyCharges
```

```
METHOD generateBill()
```

```
  daysStayed = number of days between admissionDate and today  
  IF daysStayed < 1 THEN  
    daysStayed = 1  
  ENDIF  
  billAmount = daysStayed * dailyCharges  
  RETURN billAmount
```

```
CLASS OutPatient EXTENDS Patient
```

```
    PRIVATE appointmentDate
```

```
    PRIVATE consultationFee
```

```
CONSTRUCTOR OutPatient(patientId, name, age, appointmentDate, consultationFee)
```

```
    CALL Patient CONSTRUCTOR with patientId, name, age
```

```
    SET this.appointmentDate = appointmentDate
```

```
    SET this.consultationFee = consultationFee
```

```
METHOD generateBill()
```

```
    RETURN consultationFee
```

```
MAIN
```

```
    CREATE patient1 as InPatient with wardNumber, admissionDate, dailyCharges
```

```
    CALL patient1.setDiagnosis(diagnosisText)
```

```
    CALL patient1.setPrescription(prescriptionText)
```

```
    bill1 = patient1.generateBill()
```

```
    CREATE patient2 as OutPatient with appointmentDate, consultationFee
```

```
    bill2 = patient2.generateBill()
```

```
    PRINT bill1
```

```
    PRINT bill2
```

Q8 8. E-Commerce Product Catalog [BL4 – Analyze]

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an `applyDiscount()` method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
CLASS Product
  DATA productId, name, price
  METHOD applyDiscount()
    RETURN price

CLASS Electronics EXTENDS Product
  DATA warranty, brand, voltage

  METHOD applyDiscount()
    IF warranty > 12 THEN
      discountRate ← 0.15
    ELSE
      discountRate ← 0.10
    ENDIF
    discountedPrice ← price - (price * discountRate)
    RETURN discountedPrice

CLASS Clothing EXTENDS Product
  DATA size, fabric, gender

  METHOD applyDiscount()
    IF fabric == "Cotton" THEN
      discountRate ← 0.12
    ELSE
```



```
    discountRate ← 0.08
ENDIF
discountedPrice ← price - (price * discountRate)
RETURN discountedPrice
```

CLASS Grocery EXTENDS Product

DATA expiryDate, weight

METHOD applyDiscount()

daysLeft ← difference between expiryDate and today

IF daysLeft ≤ 3 THEN

discountRate ← 0.25

ELSE IF daysLeft ≤ 7 THEN

discountRate ← 0.15

ELSE

discountRate ← 0.05

ENDIF

discountedPrice ← price - (price * discountRate)

RETURN discountedPrice

PROCEDURE Checkout

DATA cartList as list of Product

```
DATA total ← 0
```

```
FOR EACH item IN cartList DO
```

```
    total ← total + item.applyDiscount()
```

```
ENDFOR
```

```
    DISPLAY "Final Total after discounts: ", total  
END PROCEDURE
```

Q9 9. University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

Class Staff

Private name

Private staffId

Protected baseSalary

Protected department

Constructor(name, staffId, baseSalary, department)

this.name = name

this.staffId = staffId

this.baseSalary = baseSalary

this.department = department

Method calculateAllowance()

Return baseSalary * 0.05

Class TeachingStaff extends Staff

Private teachingHours

Private publications

Protected teachingFactor

Constructor(name, staffId, baseSalary, department, teachingHours, publications)

Call Staff(name, staffId, baseSalary, department)

this.teachingHours = teachingHours

this.publications = publications

```
this.teachingFactor = 0.10
```

Method calculateAllowance()

```
allowance = baseSalary * teachingFactor + (teachingHours * 50) + (publications * 100)  
Return allowance
```

Class NonTeachingStaff extends Staff

```
Private tasksCompleted  
Private roleType  
Protected nonTeachingFactor
```

Constructor(name, staffId, baseSalary, department, tasksCompleted, roleType)

```
Call Staff(name, staffId, baseSalary, department)  
this.tasksCompleted = tasksCompleted  
this.roleType = roleType  
this.nonTeachingFactor = 0.08
```

Method calculateAllowance()

```
allowance = baseSalary * nonTeachingFactor + (tasksCompleted * 25)  
Return allowance
```

Class Professor extends TeachingStaff

```
Private researchCitations  
Protected professorMultiplier
```

Constructor(name, staffId, baseSalary, department, teachingHours, publications, researchCitations)

Call TeachingStaff(name, staffId, baseSalary, department, teachingHours, publications)

this.researchCitations = researchCitations

this.professorMultiplier = 1.20

Method calculateAllowance()

baseAllow = Call TeachingStaff.calculateAllowance()

allowance = (baseAllow * professorMultiplier) + (researchCitations * 30)

Return allowance

Class Lecturer extends TeachingStaff

Private classSections

Protected lecturerMultiplier

Constructor(name, staffId, baseSalary, department, teachingHours, publications, classSections)

Call TeachingStaff(name, staffId, baseSalary, department, teachingHours, publications)

this.classSections = classSections

this.lecturerMultiplier = 1.05

Method calculateAllowance()

baseAllow = Call TeachingStaff.calculateAllowance()

allowance = (baseAllow * lecturerMultiplier) + (classSections * 200)

Return allowance

Main

staff1 = new Professor("Name1", 101, 50000, "CS", 20, 5, 10)

staff2 = new Lecturer("Name2", 102, 40000, "IT", 18, 3, 6)

staff3 = new NonTeachingStaff("Name3", 103, 35000, "Admin", 120, "Clerk")

Print staff1.calculateAllowance()

Print staff2.calculateAllowance()

Print staff3.calculateAllowance()

Q10 10. Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
 - Security implications of data hiding in SmartLock (PIN must never be exposed)
- Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

PSEUDOCODE (One combined pseudocode)

ABSTRACT CLASS SmartDevice

deviceName

CONSTRUCTOR(deviceName)

 this.deviceName = deviceName

ABSTRACT METHOD executeCommand(cmd)

CLASS SmartLight EXTENDS SmartDevice

isOn

CONSTRUCTOR(deviceName)

 super(deviceName)

 isOn = false

METHOD executeCommand(cmd)

 cmd = LOWER(cmd)

 IF cmd == "on"

 isOn = true

 PRINT deviceName + " ON"

 ELSE IF cmd == "off"

 isOn = false

 PRINT deviceName + " OFF"

 ELSE IF cmd == "status"

 IF isOn == true THEN PRINT deviceName + " ON" ELSE PRINT deviceName + " OFF"

 ELSE


```
CLASS SmartThermostat EXTENDS SmartDevice
```

```
    temperature
```

```
    heating
```

```
    CONSTRUCTOR(deviceName, initialTemp)
```

```
        super(deviceName)
```

```
        temperature = initialTemp
```

```
        heating = false
```

```
    METHOD executeCommand(cmd)
```

```
        cmd = LOWER(TRIM(cmd))
```

```
        IF cmd == "status"
```

```
            PRINT deviceName + " Temp=" + temperature + " Heating=" + heating
```

```
        ELSE IF cmd STARTS WITH "set"
```

```
            newTemp = NUMBER AFTER "set" in cmd
```

```
            temperature = newTemp
```

```
            heating = true
```

```
            PRINT deviceName + " set to " + temperature
```

```
        ELSE IF cmd == "heat"
```

```
            heating = true
```

```
            PRINT deviceName + " heating ON"
```

```
        ELSE IF cmd == "stop"
```

```
            heating = false
```

```
            PRINT deviceName + " heating STOP"
```

```
        ELSE
```

```
            PRINT "Invalid command for " + deviceName
```

```
CLASS SmartLock EXTENDS SmartDevice
```

```
    locked
```

```
    PIN (private, never printed or returned)
```

```
    CONSTRUCTOR(deviceName, givenPin)
```

```
        super(deviceName)
```

```
        locked = true
```

```
        PIN = givenPin
```

```
    METHOD executeCommand(cmd)
```

```
        cmd = LOWER(TRIM(cmd))
```

```
        IF cmd == "status"
```

```
            IF locked == true THEN PRINT deviceName + " LOCKED" ELSE PRINT deviceName + " UNLOCKED"
```

```
        ELSE IF cmd STARTS WITH "unlock"
```

```
            enteredPin = TEXT AFTER "unlock" in cmd
```

```
            IF enteredPin == PIN
```

```
                locked = false
```

```
                PRINT deviceName + " unlocked"
```

```
            ELSE
```

```
                PRINT "Unlock failed: Incorrect PIN"
```

```
        ELSE IF cmd STARTS WITH "lock"
```

```
            enteredPin = TEXT AFTER "lock" in cmd
```

```
            IF enteredPin == PIN
```

```
                locked = true
```

```
                PRINT deviceName + " locked"
```

```
            ELSE
```

```
PROGRAM SmartDeviceController
  devices = LIST of SmartDevice
  ADD new SmartLight("LivingRoomLight") to devices
  ADD new SmartThermostat("HallThermostat", 24) to devices
  ADD new SmartLock("FrontDoorLock", "1234") to devices

FOR EACH d IN devices
  d.executeCommand("status")

FOR EACH d IN devices
  d.executeCommand("on")

FOR EACH d IN devices
  d.executeCommand("unlock 1234")

FOR EACH d IN devices
  d.executeCommand("status")

END PROGRAM
```